Da Semana 3 à Semana 4

Conectando componentes, bindings e @Input/@Output ao Service, à Injeção de Dependência, aos Signals, aos Observables/RxJS e ao GitFlow

Desenvolvimento Front-End [Angular] T1 | Módulo 2 | Semana 4 | Este material não traz conteúdo novo - ele mostra, ponto a ponto, como o que você já praticou (Módulo 1 e a **Semana 3** de Angular) é a **base literal** do que vamos fazer na Semana 4. A ideia é simples: **nada aqui é assunto desconhecido, é o mesmo conhecimento crescendo.**

Como ler este material

Cada seção segue o mesmo formato: primeiro, **o que você já sabe** (com o código como ele apareceu na Semana 3); depois, **o que a Semana 4 acrescenta**; por fim, uma comparação lado a lado entre a versão "de antes" e a "de agora" do mesmo problema.

> **O fio condutor: o projeto que cresceu**
> Na Semana 3 você construiu a **biblioteca-web**: cards de livros com `@for`, botões Emprestar/Devolver e comunicação `@Input/@Output`. Agora o mesmo projeto ganha **arquitetura**. O problema-âncora é o de todo app que cresce: onde o dado mora, como a tela reage quando ele muda, como tratar o que chega ao longo do tempo e como o time trabalha junto no mesmo código?

Semana	O que você construiu	Ferramenta usada
Módulo 1	JavaScript no navegador e no Node: `localStorage`, `fetch`, `async/await`, timers, npm	`JSON.stringify/parse`, `fetch`, `setInterval`
03	Primeiro app Angular: componentes, template e comunicação pai-filho	`@for/@if/@switch`, bindings, `@Input/@Output`
04	A mesma biblioteca-web com service, estado reativo, busca em fluxo, HTTP e Git em equipe	**Assunto desta semana:** services/DI, signals, Observables/RxJS, GitFlow

CONEXÃO 1 - Do componente que faz tudo ao Service

O que você já sabe (Semana 3 e Módulo 1)

No `app.ts` da Semana 3 a lista `livrosCadastrados` e as regras de emprestar/devolver moravam **dentro do componente**. Funcionava bem, enquanto havia uma única tela.

O que a Semana 4 acrescenta

- **Service:** a classe que guarda dados e regras, fora da tela (`@Service()` no Angular 22; `@Injectable({ providedIn: 'root' })` nas versões anteriores).
- **Injeção de Dependência:** em vez de `new`, o componente **pede** o service com `inject()` e o Angular entrega **a mesma instância** (singleton).
- **Resultado:** dois componentes diferentes compartilham o mesmo dado sem `@Input/@Output` entre eles.

Comparando o código: o mesmo problema, antes e depois

SEMANA 3 - tudo no componente

```
export class App {
  livros: Livro[] = [ /* ... */ ];
  emprestar(id: number) {
    const l = this.livros.find(x => x.id === id);
    if (l) l.disponivel = false;
  }
}
```

SEMANA 4 - dados e regras no service

```
// livro-service.ts
@Service()
export class LivroService {
  private _livros = signal<Livro[]>([ /* ... */ ]);
  readonly livros = this._livros.asReadonly();
  emprestar(id: number) { /* update + map */ }
}
// app.ts
export class App {
  private service = inject(LivroService);
  livros = this.service.livros;   // template:
livros()
}
```

> **O que realmente mudou?**
> A lógica é a mesma. Muda **onde ela mora** (service, reaproveitável) e **quem cria o objeto** (o Angular, não você).

CONEXÃO 2 - Da interpolação e do binding aos signals

O que você já sabe (Semana 3 e Módulo 1)

Você usou `{{ }}`, `[propriedade]` e `(evento)` para ligar a classe ao template. O Angular "descobria" o que mudou verificando a tela depois de cada evento.

O que a Semana 4 acrescenta

- **Signal:** um valor que **avisa quem o lê** quando muda (como a célula de uma planilha).
- **`computed`:** valor calculado a partir de outros signals (a fórmula).
- **`effect`:** efeito colateral quando algo muda (salvar, logar).
- No template, signals são lidos **com parênteses**: `livros()`.

Comparando o código: o mesmo problema, antes e depois

SEMANA 3

```
livros: Livro[] = [...];
// template
@for (livro of livros; track livro.id) { ... }
// total recalculado a cada verificação
total() { return this.livros.length; }
```

SEMANA 4

```
livros = signal<Livro[]>([...]);
total = computed(() => this.livros().length);
// template
@for (livro of livros(); track livro.id) { ... }
```

O que realmente mudou?

O template continua igual no **espírito**. A diferença é que agora a dependência é **explícita**: o Angular sabe exatamente o que depende de `livros`.

CONEXÃO 3 - Do @Input/@Output e do two-way binding ao input(), output() e model()

O que você já sabe (Semana 3 e Módulo 1)

Na Semana 3 o pai enviava dados com `@Input()` e o filho avisava com `@Output() + EventEmitter`. O two-way combinava `[ ]` e `( )`.

O que a Semana 4 acrescenta

- `input()` e `input.required()` entregam o valor **como signal**.
- `output()` substitui `@Output() + new EventEmitter`.
- `model()` cria **entrada + saída** em um só signal: `[(termo)]="busca"`.
- Os decorators **continuam funcionando**; a API nova se integra com `computed`.

Comparando o código: o mesmo problema, antes e depois

SEMANA 3 - decorators

```
@Input({ required: true }) livro!: Livro;
@Output() emprestar = new EventEmitter<number>();
// template: {{ livro.titulo }}
```

SEMANA 4 - API de signals

```
livro = input.required<Livro>();
emprestar = output<number>();
// template: {{ livro().titulo }}
```

> **O que realmente mudou?**
> O **pai não muda** (`[livro]="x"` e `(emprestar)="f($event)"`). Só o filho troca a forma de declarar.

CONEXÃO 4 - Do localStorage, do fetch e dos timers aos Observables

O que você já sabe (Semana 3 e Módulo 1)

No Módulo 1 você usou `fetch` com `await` (uma resposta, uma vez), `setInterval` (algo repetido no tempo) e `localStorage` com `JSON.stringify/parse`. Também aprendeu a **limpar** timers com `clearInterval`.

O que a Semana 4 acrescenta

- **Observable:** uma sequência de valores **ao longo do tempo**; só executa com `subscribe`.
- `HttpClient` devolve Observables, com `retry`, `catchError`, `finalize`.
- `interval(1000)` é o `setInterval` com operadores e cancelamento (`unsubscribe`).
- `effect` + `localStorage` mantém o estado salvo **sozinho** a cada mudança.

Comparando o código: o mesmo problema, antes e depois

MÓDULO 1 - timers e fetch

```
const id = setInterval(() => console.log('tick'),
1000);
// ...
clearInterval(id);   // limpar à mão
const r = await fetch('livros.json');
const livros = await r.json();
```

SEMANA 4 - Observables

```
const sub = interval(1000).subscribe(() =>
console.log('tick'));
// ...
sub.unsubscribe();   // ou take(5) /
takeUntilDestroyed()
this.http.get<Livro[]>('livros.json')
  .pipe(retry(2), catchError(() => of([])))
  .subscribe(l => this.livros = l);
```

> **O que realmente mudou?**
> O raciocínio ("agendar, receber, limpar") é o mesmo. O RxJS dá **nomes, operadores e cancelamento** para o que você já fazia à mão.

CONEXÃO 5 - Do git add/commit ao fluxo de equipe (GitFlow)

O que você já sabe (Semana 3 e Módulo 1)

Se você já usou `git add`, `git commit` e `git push` sozinho, sabe guardar versões do seu código. Em equipe, o desafio muda: **duas pessoas no mesmo arquivo**.

O que a Semana 4 acrescenta

- **Branches** com função: `main`, `develop`, `feature/*`, `release/*`, `hotfix/*`.
- **Pull Request**: o time revisa antes de aceitar.
- **Conflito** de merge: normal; você escolhe o resultado final.
- **Conventional Commits** e **tags SemVer** (`v1.0.0`) - o mesmo `MAIOR.MENOR.CORREÇÃO` do `^4.1.2` do npm.

Comparando o código: o mesmo problema, antes e depois

SOZINHO

```
git add .
git commit -m "mudei coisas"
git push origin main
```

EM EQUIPE

```
git switch -c feature/busca develop
git commit -m "feat(busca): filtra por categoria"
git push -u origin feature/busca
# abrir Pull Request para develop -> revisão ->
merge
```

> **O que realmente mudou?**
> Os comandos são os mesmos; muda o **combinado do time**: onde se trabalha, como se revisa e como se versiona.

Checklist: você já está pronto para a Semana 4 se...

- ✓ Tem o projeto **biblioteca-web** da Semana 3 funcionando (ou usará a pasta `Ex0 - Base`).
- ✓ Sabe usar `@for`, `@if`, `[propriedade]`, `(evento)` e `{{ }}`.
- ✓ Lembra de `@Input/@Output` e de como o pai conversa com o filho.
- ✓ Sabe rodar `ng serve`, `ng generate` e ler uma mensagem de erro inteira.
- ✓ Lembra que `localStorage` só guarda texto (`JSON.stringify/parse`) e o que `fetch` e `async/await` faziam.
- ✓ Tem conta no **GitHub** e o **Git** instalado (`git --version`).
- ✓ Tem Node compatível com o Angular 22 (`node --version` deve ser ^22.22.3 ou mais novo; confira `ng version`).

Se todos esses pontos fazem sentido, a Semana 4 não apresenta nada desconhecido - ela dá nome, comando e ferramenta profissional (service, signals, RxJS, GitFlow) para instintos que você já desenvolveu.